

ГЛАВА 14

ПЕРЕГРУЗКА ОПЕРАТОРОВ И ПРЕОБРАЗОВАНИЙ

В этой главе...

14.1. Фундаментальные концепции	700
14.2. Операторы ввода и вывода	705
14.3. Арифметические операторы и операторы отношения	710
14.4. Операторы присвоения	713
14.5. Оператор индексирования	715
14.6. Операторы инкремента и декремента	717
14.7. Операторы доступа к членам	720
14.8. Оператор вызова функции	722
14.9. Перегрузка, преобразование и операторы	732
Резюме	744
Термины	745

Как упоминалось в главе 4, язык C++ предоставляет для встроенных типов множество операторов и автоматических преобразований. Они позволяют создавать разнообразные выражения, где используются разные типы данных.

Язык C++ позволяет переопределять смысл операторов, применяемых для объектов типа класса, а также определять для класса функции преобразования типов. Функции преобразования типа класса используются подобно встроенным преобразованиям для неявного преобразования (при необходимости) объекта одного типа в другой.

Перегрузка оператора (overloaded operator) позволяет определить смысл оператора, когда он применяется к операнду (операндам) типа класса. Разумное применение перегрузки операторов способно упростить программы, облегчить их написание и чтение. Например, поскольку наш первоначальный класс `Sales_item` (см. раздел 1.5.1, стр. 47) определял операторы ввода, вывода и суммы, сумму двух объектов класса `Sales_item` можно вывести так:

```
cout << item1 + item2; // вывод суммы двух объектов класса Sales_item
```

Класс `Sales_data` (см. раздел 7.1, стр. 332), напротив, еще не имеет перегруженных операторов, поэтому код вывода суммы его объектов окажется более подробным, а следовательно, менее ясным:

```
print(cout, add(data1, data2)); // вывод суммы двух объектов
                                // класса Sales_data
```



14.1. Фундаментальные концепции

Перегруженный оператор — это функция со специальным именем, состоящим из ключевого слова `operator`, сопровождаемого символом определяемого оператора. Подобно любой другой функции, перегруженный оператор имеет тип возвращаемого значения и список параметров.

Количество параметров функции перегруженного оператора совпадает с количеством операндов оператора. У унарного оператора — один параметр; у бинарного — два. В бинарном операторе левый операнд передается первому параметру, а правый операнд — второму. За исключением перегруженного оператора вызова функции, `operator()`, у перегруженного оператора не может быть аргументов по умолчанию (см. раздел 6.5.1, стр. 308).

Если перегруженный оператор является функцией-членом, то первый (левый) операнд связывается с неявным указателем `this` (см. раздел 7.1.2, стр. 336). Поскольку первый операнд неявно связан с указателем `this`, функция оператора-члена будет иметь на один явный параметр меньше, чем операндов у оператора.



Когда перегруженный оператор является функцией-членом, указатель `this` соответствует левому операнду. У операторов-членов на один явный параметр меньше, чем операндов.

Функция оператора должна быть либо членом класса, либо иметь по крайней мере один параметр типа класса:

```
// ошибка: переопределить встроенный оператор для целых чисел
int operator+(int, int);
```

Это ограничение означает невозможность изменить смысл оператора, относящегося к операндам встроенного типа.

Перегрузить можно многие, но не все операторы. Табл. 14.1 демонстрирует, может ли оператор быть перегружен. Перегрузка операторов `new` и `delete` рассматривается в разделе 19.1.1 (стр. 1024).

Перегрузить можно только существующие операторы и нельзя изобрести новые символы операторов. Например, нельзя определить оператор `operator**` для возведения числа в степень.

Таблица 14.1. Операторы

Операторы, которые могут быть перегружены					
+	-	*	/	%	^
&		~	!	,	=
<	>	<=	>=	++	--
<<	>>	==	!=	&&	
+=	-=	/=	%=	^=	&=
=	*=	<<=	>>=	[]	()
->	->*	new	new []	delete	delete []
Операторы, которые не могут быть перегружены					
:: .* . ?:					

Четыре символа (+, -, * и &) служат и унарными операторами, и бинарными. Перегружен может быть один или оба из этих операторов. Определяемый оператор задает количество параметров:

```
x == y + z;
```

Это будет эквивалентно `x == (y + z)`.

Непосредственный вызов функции перегруженного оператора

Обычно функцию перегруженного оператора вызывают косвенно, применив оператор к аргументам соответствующего типа. Но функцию перегруженного оператора можно также вызвать непосредственно, как обычную функцию. Достаточно указать имя функции и передать соответствующее количество аргументов соответствующего типа:

```
// эквивалент вызова функции оператора, не являющегося членом класса
data1 + data2;           // обычное выражение
operator+(data1, data2); // эквивалентный вызов функции
```

Эти вызовы эквивалентны: оба они являются вызовом функции не члена класса `operator+()` с передачей `data1` как первого аргумента и `data2`, так и второго.

Явный вызов функции оператора-члена осуществляется таким же образом, как и вызов любой другой функции-члена: имя объекта (или указателя), для которого выполняется функция, и оператор точки (или стрелки) для выбора функции, которую следует вызвать:

```
data1 += data2;           // вызов на базе выражения
data1.operator+=(data2); // эквивалентный вызов функции оператора-члена
```

Каждый из этих операторов вызывает функцию-член `operator+=`, где указатель `this` содержит адрес объекта `data1`, а объект `data2` передан как аргумент.

Некоторые операторы не следует перегружать

Помните, что некоторые операторы гарантируют порядок вычисления операндов. Поскольку использование перегруженного оператора на самом деле является вызовом функции, эти гарантии не распространяются на перегруженные операторы. В частности, гарантии вычисления операндов логических операторов `AND` и `OR` (см. раздел 4.3, стр. 197), оператора запятой (см. раздел 4.10, стр. 217) не сохраняются. Кроме того, перегруженные версии операторов `&&` и `||` не поддерживают вычисления по сокращенной схеме. Оба операнда вычисляются всегда.

Поскольку перегруженные версии этих операторов не сохраняют порядок вычисления и (или) не поддерживают вычисления по сокращенной схеме, их перегрузка обычно — плохая идея. Пользователи, вероятно, будут удивлены отсутствием привычных гарантий последовательности вычисления в коде при использовании перегруженной версии одного из этих операторов.

Еще один повод не перегружать операторы запятой и обращения к адресу заключается в том, что, в отличие от большинства операторов, язык сам определяет значение этих операторов, когда они применены к объектам типа класса. Поскольку у этих операторов есть встроенное значение, они обычно не должны перегружаться. Пользователи класса будут удивлены, если они поведут себя не так, как обычно.



Обычно операторы запятой, обращение к адресу, логический оператор `AND` и `OR` *не должны* быть перегружены.

Использование определений, совместимых со встроенным смыслом

При разработке класса всегда следует сначала подумать об обеспечиваемых им операциях. Только определившись с необходимыми операциями, следует подумать о том, стоит ли определить некую операцию как обычную функцию или как перегруженный оператор. Те операции, которые логически соответствуют операторам, — это хорошие кандидаты на определение в качестве перегруженных операторов.

- Если класс осуществляет операции ввода и вывода, имеет смысл определить операторы сдвига для совместимости с таковыми у встроенных типов.
- Если класс подразумевает проверку на равенство, определите оператор `operator==`. Если у класса есть оператор `operator==`, то у него обычно должен быть также оператор `operator!=`.

- Если у класса должна быть операция упорядочивания, определите оператор `operator<`. Если у класса есть оператор `operator<`, то у него, вероятно, должны быть все операторы сравнения.
- Тип возвращаемого значения перегруженного оператора обычно должен быть совместимым с таковым у встроенной версии оператора: логические операторы и операторы отношения должны возвращать значение типа `bool`, арифметические операторы должны возвращать значение типа класса, операторы присвоения и составные операторы присвоения должны возвращать ссылку на левый операнд.

Составные операторы присвоения

Операторы присвоения должны вести себя аналогично синтезируемым операторам: после присвоения значения левых и правых операндов должны быть одинаковы, а возвратить оператор должен ссылку на левый операнд. Перегруженный оператор присвоения должен обобщить смысл встроенного оператора присвоения, а не переименовывать его.

ВНИМАНИЕ! БУДЬТЕ ОСТОРОЖНЫ ПРИ ИСПОЛЬЗОВАНИИ ПЕРЕГРУЖЕННЫХ ОПЕРАТОРОВ

Каждый оператор имеет некий смысл, когда он используется для встроенных типов. Бинарный оператор `+`, например, всегда означает сумму. Вполне логично и удобно применять в классе бинарный оператор `+` для аналогичной функции. Например, библиотечный тип `string`, в соответствии с соглашением, общепринятым для множества языков программирования, использует оператор `+` для конкатенации, т.е. добавления содержимого одной строки в другую.

Перегруженные операторы полезней всего тогда, когда смысл встроенного оператора логически соответствует функции текущего класса. Применение перегруженных операторов вместо именованных функций позволяет сделать программы более простыми, естественными и интуитивно понятными. Злоупотребление перегруженными операторами, а также придание не свойственного им смысла сделает класс неудобным в применении.

На практике вполне очевидные случаи противоестественной перегрузки операторов довольно редки. Например, ни один ответственный программист не переопределил бы оператор `operator+` для вычитания. Зато очень часто предпринимаются попытки неким образом приспособить “обычный” оператор, который неприменим к данному классу. Операторы следует использовать только для тех функций, которые будут однозначно поняты пользователями. Оператор с неоднозначным смыслом, например равенство, может быть интерпретирован по-разному.

Если класс обладает арифметическим (см. раздел 4.2, стр. 194) или побитовым (см. раздел 4.8, стр. 210) оператором, то его, как правило, имеет смысл снабдить соответствующими составными операторами. Вполне логично было бы также определить и оператор `+=`. Само собой разумеется, оператор `+=` должен быть определен так, чтобы он вел себя аналогично встроенным операторам, т.е. осуществлял составное присвоение: сначала сумма (`+`), а затем присвоение (`=`).

Выбор обычной функции или члена класса

При проектировании перегруженных операторов необходимо принять решение, должен ли каждый из них быть членом класса или обычной функцией (не членом класса). В некоторых случаях выбора нет; оператор должен быть членом класса. В других случаях можно принять во внимание несколько эмпирических правил, которые помогут принять решение.

Приведенный ниже список критериев может оказаться полезен в ходе принятия решения о том, следует ли сделать оператор функцией-членом класса или обычной функцией.

- Операторы присвоения (`=`), индексирования (`[]`), вызова (`()`) и доступа к члену класса (`->`) *следует* определять как функции-члены класса.
- Подобно оператору присвоения, составные операторы присвоения *обычно* должны быть членами класса. Но в отличие от оператора присвоения, это не обязательно.
- Другие операторы, которые изменяют состояние своего объекта или жестко связаны с данным классом (например, инкремент, декремент и обращение к значению), *обычно* должны быть членами класса.
- Симметричные операторы, такие как арифметические, операторы равенства, операторы сравнения и побитовые операторы, лучше определять как обычные функции, а не члены класса.

Разработчики ожидают возможности использовать симметричные операторы в выражениях со смешанными типами. Например, возможности сложить переменные типа `int` и `double`. Сложение симметрично, а потому можно использовать тип как левого, так и правого операнда.

Если необходимо обеспечить подобные выражения смешанного типа, действующие объекты класса, то оператор должен быть определен как функция, не являющаяся членом класса.

При определении оператора как функции-члена левый операнд должен быть объектом того класса, членом которого является этот оператор. Например:

```
string s = "world";  
string t = s + "!"; // ok: const char* можно добавить к строке
```

```
string u = "hi" + s; // возможна ошибка, если + будет членом
                     // класса string
```

Если бы оператор `operator+` был членом класса `string`, то первый случай сложения был бы эквивалентен `s.operator+("!")`. Аналогично сложение `"hi" + s` было бы эквивалентно `"hi".operator+(s)`. Однако литерал `"hi"` имеет тип `const char*`, т.е. встроенный тип; у него нет функций-членов.

Поскольку класс `string` определяет оператор `+` как обычную функцию, не являющуюся членом класса, сложение `"hi" + s` эквивалентно вызову `operator+("hi", s)`. Подобно любому вызову функции, каждый из аргументов должен быть преобразуем в тип параметра. Единственное требование — по крайней мере один из операндов должен иметь тип класса, а оба операнда могут быть преобразованы в строку.

Упражнения раздела 14.1

Упражнение 14.1. Чем перегруженный оператор отличается от встроенного? В чем перегруженные операторы совпадают со встроенными?

Упражнение 14.2. Напишите объявления для перегруженных операторов ввода, вывода, сложения и составного присвоения для класса `Sales_data`.

Упражнение 14.3. Классы `string` и `vector` определяют перегруженный оператор `==`, применимый для сравнения объектов этих типов. Если векторы `svec1` и `svec2` содержат строки, объясните, какая из версий оператора `==` применяется в каждом из следующих выражений:

- (a) `"cobble" == "stone"` (b) `svec1[0] == svec2[0]`
(c) `svec1 == svec2` (d) `"svec1[0] == "stone"`

Упражнение 14.4. Объясните, должен ли каждый из следующих операторов быть членом класса и почему?

- (a) `%` (b) `%=` (c) `++` (d) `->` (e) `<<` (f) `&&` (g) `==` (h) `()`

Упражнение 14.5. В упражнении 7.40 из раздела 7.5.1 (стр. 377) был приведен набросок одного из следующих классов. Какой из перегруженных операторов должен (если должен) предоставить класс.

- (a) `Book` (b) `Date` (c) `Employee`
(d) `Vehicle` (e) `Object` (f) `Tree`



14.2. Операторы ввода и вывода

Как уже упоминалось, библиотека `IO` использует операторы `>>` и `<<` для ввода и вывода соответственно. Сама библиотека `IO` определяет версии этих операторов для ввода и вывода данных встроенных типов. Классы, нуждающиеся во вводе и выводе, обычно определяют версии этих операторов для объектов данного класса.



14.2.1. Перегрузка оператора вывода <<

Обычно первый параметр оператора вывода является ссылкой на неконстантный объект класса `ostream`. Объект класса `ostream` неконстантен потому, что запись в поток изменяет его состояние. Параметр является ссылкой потому, что нельзя копировать объект класса `ostream`.

Второй параметр обычно должен быть ссылкой на константу типа класса, объект которого необходимо вывести. Параметр должен быть ссылкой во избежание копирования аргумента. Но он может быть константной ссылкой потому, что вывод объекта обычно не изменяет его.

Для совместимости с другими операторами вывода оператор `operator<<` обычно возвращает свой параметр типа `ostream`.

Оператор вывода класса `Sales_data`

Для примера напишем оператор вывода для класса `Sales_data`:

```
ostream &operator<<(ostream &os, const Sales_data &item)
{
    os << item.isbn() << " " << item.units_sold << " "
        << item.revenue << " " << item.avg_price();
    return os;
}
```

За исключением имени эта функция идентична прежней версии функции `print()` (см. раздел 7.1.3, стр. 340). Вывод объекта класса `Sales_data` требует вывода значений всех его трех переменных-членов, а также вычисления средней цены (`average price`). Каждый элемент отделяется пробелом. После вывода значений оператор возвращает ссылку на использованный для этого объект класса `ostream`.

Операторы вывода обеспечивают минимум форматирования

Операторы вывода встроенных типов форматирования практически не обеспечивают. В частности, они не выводят символ новой строки. Пользователи ожидают, что операторы вывода класса будут вести себя так же. Если бы оператор выводил новую строку, то пользователь не смог бы вывести содержимое объекта с описывающим его текстом в одной строке. Оператор вывода, обеспечивающий минимум форматирования, позволяет контролировать подробности вывода пользователям.



Обычно операторы вывода должны выводить содержимое объекта с минимальным форматированием. Они не должны выводить новую строку.

Операторы ввода-вывода не должны быть функциями-членами класса

Операторы ввода и вывода, соответствующие соглашениям библиотеки `iostream`, должны быть обычными функциям, а не членами класса. Эти операторы не могут быть членами нашего класса. Если бы это было так, то левый операнд должен был быть объектом типа нашего класса:

```
Sales_data data;  
data << cout; // если бы оператор operator<< был членом класса Sales_data
```

Если бы эти операторы были членами некоего класса, то они должны были бы быть членами класса `istream` или `ostream`. Но эти классы являются частью стандартной библиотеки, а добавлять члены в библиотечные классы нельзя.

Таким образом, если необходимо определить операторы ввода-вывода для собственного типа, их следует определить как функции, не являющиеся членами класса. Конечно, операторы ввода-вывода обычно должны читать или выводить данные не открытых переменных-членов. Как следствие, операторы ввода-вывода обычно объявляют дружественными (см. раздел 7.2.1, стр. 351).

Упражнения раздела 14.2.1

Упражнение 14.6. Определите оператор вывода для класса `Sales_data`.

Упражнение 14.7. Определите оператор вывода для класса `String`, написанного для упражнений раздела 13.5 (стр. 671).

Упражнение 14.8. Определите оператор вывода для класса, который был выбран в упражнении 7.40 раздела 7.5.1 (стр. 377).

14.2.2. Перегрузка оператора ввода >>

Обычно первый параметр оператора ввода является ссылкой на поток, из которого осуществляется чтение, а второй параметр — ссылкой на некий неконстантный объект, в который предстоит прочитать данные. Обычно оператор возвращает ссылку на свой поток. Второй параметр не должен быть константным потому, что задачей оператора ввода и является собственно запись данных в этот объект.

Оператор ввода класса `Sales_data`

В качестве примера напишем оператор ввода для класса `Sales_data`:

```
istream &operator>>(istream &is, Sales_data &item)  
{  
    double price; // инициализировать не нужно; читать в price  
                  // прежде, чем использовать
```

```

is >> item.bookNo >> item.units_sold >> price;
if (is)          // проверить успех ввода данных
    item.revenue = item.units_sold * price;
else
    item = Sales_data(); // ввод неудачен: вернуть объект в
                        // стандартное состояние
return is;
}

```

За исключением оператора `if` это определение подобно прежней функции `read()` (см. раздел 7.1.3, стр. 340). Оператор `if` проверяет, было ли чтение успешно. Если произойдет ошибка ввода-вывода, он вернет объект `Sales_data` в состоянии пустого объекта. Это гарантирует корректность состояния объекта.



Операторы ввода должны учитывать возможность неудачи ввода, а операторы вывода об этом могут не заботиться.

Ошибки во время ввода

В операторе ввода возможны следующие ошибки.

- Операция чтения может потерпеть неудачу из-за наличия в потоке данных неподходящего типа. Например, после чтения переменной-члена `bookNo` оператор ввода подразумевает, что следующие два элемента будут числовыми данными. Если во вводе окажутся не числовые данные, поток будет недопустим и все последующее попытки чтения из него потерпят неудачу.
- Во время любой из операций чтения может встретиться конец файла или произойти другая ошибка потока ввода.

Чтобы не проверять каждую часть прочитанных данных, можно проверить состояние потока в целом и только потом использовать прочитанные данные

```

if (is) // проверить успех ввода данных
    item.revenue = item.units_sold * price;
else
    item = Sales_data(); // ввод неудачен: вернуть объект в
                        // стандартное состояние

```

При сбое любой из операций чтения значение переменной-члена `price` останется неопределенным. Следовательно, перед ее использованием следует проверить, допустим ли еще поток ввода. Если это так, осуществляется вычисление значения переменной `revenue`. В случае ошибки ничего страшного не произойдет, поскольку будет возвращен пустой объект класса `Sales_data`. Для этого объекту `item` присваивается новый объект класса `Sales_data`, созданный при помощи стандартного конструктора. После этого присвоения переменная-член `bookNo` объекта `item` будет содержать пустую строку, а его переменные члены `revenue` и `units_sold` — нулевое значение.

Возвращение объекта в допустимое состояние особенно важно, если объект мог быть частично изменен прежде, чем произошла ошибка. Например, в данном операторе ввода ошибка могла бы произойти уже после успешного чтения в переменную-член `bookNo`. В результате значения переменных-членов `units_sold` и `revenue` останутся неизменными. Таким образом, новое значение `bookNo` будет связано с данными прежнего объекта.

Оставляя объект в допустимом состоянии, можно в некоторой степени защитить пользователя, который игнорирует возможность ошибки ввода. Объект будет находиться в пригодном для использования состоянии — все его члены окажутся определены. Кроме того, объект не будет вводить в заблуждение — его данные останутся единообразными.



Проектируя оператор ввода, очень важно решить, что делать в случае ошибки и как вновь сделать объект доступным.

Оповещение об ошибке

Некоторые операторы ввода нуждаются в дополнительной проверке данных. Например, оператор ввода мог бы проверить соответствие формату данных, читаемых в переменную `bookNo`. В таких случаях оператору ввода возможно понадобится установить флаг состояния потока так, чтобы он означал отказ (см. раздел 8.1.2, стр. 402), хотя с технической точки зрения чтение было успешно. Обычно оператор ввода устанавливает только флаг `failbit`. Флаг `eofbit` подразумевал бы конец файла, а бит `badbit` — нарушение потока. Установку этих флагов лучше оставить библиотеке IO.

Упражнения раздела 14.2.2

Упражнение 14.9. Определите оператор ввода для класса `Sales_data`.

Упражнение 14.10. Опишите поведение оператора ввода класса `Sales_data` при следующем вводе:

(a) 0-201-99999-9 10 24.95 (b) 10 24.95 0-210-99999-9

Упражнение 14.11. Что не так со следующим оператором ввода класса `Sales_data`? Что будет при передаче этому оператору данных предыдущего упражнения?

```
istream& operator>>(istream& in, Sales_data& s)
{
    double price;
    in >> s.bookNo >> s.units_sold >> price;
    s.revenue = s.units_sold * price;
    return in;
}
```

Упражнение 14.12. Определите оператор ввода для класса, использованного в упражнении 7.40 раздела 7.5.1 (стр. 377). Обеспечьте обработку оператором ошибок ввода.

14.3. Арифметические операторы и операторы отношения

Как правило, арифметические операторы и операторы отношения определяют как функции не члены класса, чтобы обеспечить преобразования и для левого, и для правого операнда (см. раздел 7.1.5, стр. 704). Эти операторы не должны изменять состояние любого из операндов, поэтому их параметры обычно являются ссылками на константу.

Обычно арифметический оператор создает новое значение, являющееся результатом вычисления двух своих операндов. Это значение отлично от каждого из операндов и вычисляется в локальной переменной. Оператор возвращает как результат копию этого локального значения. Классы, определяющие арифметический оператор, определяют также соответствующий составной оператор присвоения. Когда у класса есть два оператора, как правило, эффективней определять арифметический оператор для составного присвоения:

```
// подразумевается, что оба объекта относятся к той же книге
Sales_data
operator+(const Sales_data &lhs, const Sales_data &rhs)
{
    Sales_data sum = lhs; // копирование переменных-членов из lhs в sum
    sum += rhs;           // добавить rhs к sum
    return sum;
}
```

Это определение очень похоже на оригинальную функцию `add()` (см. раздел 7.1.3, стр. 340). Значение `lhs` копируется в локальную переменную `sum`. Затем оператор составного присвоения класса `Sales_data` (определенный на стр. 715) добавляет значение `rhs` к `sum`. Функция завершает работу, возвращая копию значения переменной `sum`.



Классы, в которых определен арифметический оператор и соответствующий ему составной оператор, обычно реализуют арифметический оператор при помощи составного.

Упражнения раздела 14.3

Упражнение 14.13. Какие еще арифметические операторы (см. табл. 4.1, стр. 194), если таковые вообще есть, должны, по-вашему, поддержать класс `Sales_data`? Определите эти операторы.

Упражнение 14.14. Почему оператор `operator+` эффективней определять как вызывающий оператор `operator+=`, а не наоборот?

Упражнение 14.15. Должен ли класс, выбранный в упражнении 7.40 раздела 7.5.1 (стр. 377), определять какие-либо арифметические операторы? Если да, то реализуйте их. В противном случае объясните, почему нет.



14.3.1. Операторы равенства

Классы языка C++ используют оператор равенства для проверки эквивалентности объектов. Он сравнивает каждую переменную-член обоих объектов и признает их равными, если все значения одинаковы. В соответствии с этой концепцией оператор равенства класса `Sales_data` должен сравнить переменные `bookNo` двух объектов, а также значения их остальных переменных.

```
bool operator==(const Sales_data &lhs, const Sales_data &rhs)
{
    return lhs.isbn() == rhs.isbn() &&
           lhs.units_sold == rhs.units_sold &&
           lhs.revenue == rhs.revenue;
}
bool operator!=(const Sales_data &lhs, const Sales_data &rhs)
{
    return !(lhs == rhs);
}
```

Определение этих функций тривиально. Однако важнее всего принципы, которые здесь используются.

- Если в классе определен оператор, позволяющий выяснить равенство двух объектов данного класса, его функция должна иметь имя `operator==`. Не стоит изобретать для нее другое имя, поскольку пользователи ожидают, что для сравнения объектов можно использовать именно оператор `==`. Кроме того, это гораздо проще, чем каждый раз запоминать новые имена.
- Если в классе определен оператор `==`, то два объекта могут содержать одинаковые данные.
- Обычно оператор равенства должен быть транзитивным, т.е. если оба выражения, `a == b` и `b == c`, являются истинными, то `a == c` тоже должно быть истиной.
- Если в классе определен оператор `operator==`, следует также определить и оператор `operator!=`. Пользователи вполне резонно будут полагать, что если применимо равенство, то применимо и неравенство.
- Определяя операторы равенства и неравенства, почти всегда имеет смысл использовать один из них для создания другого. Один оператор должен фактически сравнивать объекты, а второй — использовать его в своих целях.



Классы, в которых определен оператор `operator==`, гораздо проще использовать со стандартной библиотекой. Если оператор `==` определен в классе, то такие алгоритмы к нему можно применять без всякой дополнительной подготовки.

Упражнения раздела 14.3.1

Упражнение 14.16. Определите операторы равенства и неравенства для классов `StrBlob` (см. раздел 12.1.1, стр. 582), `StrBlobPtr` (см. раздел 12.1.6, стр. 604), `StrVec` (см. раздел 13.5, стр. 666) и `String` (см. раздел 13.5, стр. 671).

Упражнение 14.17. Должен ли класс, выбранный в упражнении 7.40 раздела 7.5.1 (стр. 377), определять операторы равенства? Если да, то реализуйте их. В противном случае объясните, почему нет.



14.3.2. Операторы отношения

Классы, для которых определен оператор равенства, зачастую (но не всегда) обладают операторами отношения. В частности, это связано с тем, что ассоциативные контейнеры и некоторые из алгоритмов используют оператор меньше (`operator<`).

Обычно операторы отношения должны определять следующее.

1. Порядок отношений, совместимый с требованиями для ключей ассоциативных контейнеров (см. раздел 11.2.2, стр. 543);
2. Отношение, совместимое с равенством, если у класса есть оба оператора. В частности, если два объекта не равны, то один объект должен быть меньше другого.



Вполне резонно предположить, что класс `Sales_data` должен поддерживать операторы отношения, хотя это и не обязательно. Причины не столь очевидны, поэтому рассмотрим их подробнее.

Можно подумать, что оператор `<` будет определен так же, как функция `compareIsbn()` (см. раздел 11.2.2, стр. 543). Эта функция сравнивала объекты класса `Sales_data` за счет сравнения их ISBN. Хотя функция `compareIsbn()` обеспечивает порядок отношений, что соответствует первому требованию, она возвращает результат, противоречащий определению равенства. В результате она не удовлетворяет второму требованию.

Оператор `==` класса `Sales_data` считает две транзакции с одинаковым ISBN неравными, если у них отличаются значения переменных-членов `revenue` или `units_sold`. Если бы оператор `<` был определен как сравнивающий только значения ISBN, то два объекта с одинаковым ISBN, но разными `units_sold` или `revenue` считались бы неравными, но ни один из объектов не был бы меньше

другого. Как правило, если имеются два объекта, ни один из которых не меньше другого, то вполне логично ожидать, что эти объекты равны.

Создается впечатление, что имеет смысл определить оператор `operator<` для сравнения каждой переменной-члена по очереди. Его можно было бы определить так, чтобы при равных `isbn` объекты сравнивались по переменной-члену `units_sold`, а затем `revenue`.

Однако никаких оснований для упорядочивания здесь нет. В зависимости от того, как планируется использовать класс, определить порядок можно сначала на основании переменных `revenue` и `units_sold`. Можно было бы установить, что объекты с меньшим значением переменной `units_sold` были “меньше”, чем таковые с большим. Либо можно было бы установить, что объекты с меньшим значением переменной-члена `revenue` “меньше”, чем таковые с большим значением.

Для класса `Sales_data` нет единого логического определения значения “меньше”. Таким образом, для этого класса лучше вообще не определять оператор `operator<`.



Если есть однозначное логическое определение значения “меньше”, то классы обычно должны определять оператор `operator<`. Но если у класса есть также оператор `operator==`, то определяйте оператор `operator<`, только если определения смысла понятий “меньше” и “равно” не противоречат друг другу.

Упражнения раздела 14.3.2

Упражнение 14.18. Определите операторы отношения для классов `StrBlob`, `StrBlobPtr`, `StrVec` и `String`.

Упражнение 14.19. Определяет ли класс, выбранный в упражнении 7.40 раздел 7.5.1 (стр. 377), операторы отношения? Если да, то реализуйте их. В противном случае объясните, почему нет.

14.4. Операторы присвоения

Кроме операторов присвоения копии и присваивания при перемещении, которые присваивают один объект типа класса другому объекту того же класса (см. раздел 13.1.2, стр. 634 и раздел 13.6.2, стр. 679), в классе можно определить дополнительные операторы присвоения, позволяющие использовать в качестве правого операнда другие типы.

Например, библиотечный класс `vector`, кроме операторов присвоения копии и присваивания при перемещении, определяет третий оператор присвоения, получающий заключенный в фигурные скобки список элементов (см. раздел 9.2.5, стр. 435). Этот оператор можно использовать следующим образом:

```
vector<string> v;
v = {"a", "an", "the"};
```

Такой оператор можно также добавить в класс `StrVec` (см. раздел 13.5, стр. 666):

```
class StrVec {
public:
    StrVec &operator=(std::initializer_list<std::string>);
    // другие члены, как в разделе 13.5 (стр. 666)
};
```

Чтобы не отличаться от операторов присвоения для встроенных типов (и уже определенных операторов присвоения копии и присваивания при перемещении), новый оператор присвоения будет возвращать ссылку на левый операнд:

```
StrVec &StrVec::operator=(initializer_list<string> il)
{
    // alloc_n_copy() резервирует пространство и копирует элементы
    // из заданного диапазона
    auto data = alloc_n_copy(il.begin(), il.end());
    free(); // удалить элементы в этом объекте и освободить пространство
    elements = data.first; // обновить переменные-члены, чтобы указывать
                          // на новое пространство
    first_free = cap = data.second;
    return *this;
}
```

Подобно операторам присвоения копии и присваивания при перемещении, другие перегруженные операторы присвоения должны освобождать существующие элементы и создавать новые. В отличие от операторов копирования и присваивания при перемещении, этот оператор не должен проверять случай присвоения себя себе. Параметр имеет тип `initializer_list<string>` (см. раздел 6.2.6, стр. 290), а это означает, что объект `il` не может быть тем же объектом, на который указывает указатель `this`.



Операторы присвоения могут быть перегружены. Независимо от типа параметра, операторы присвоения следует определять как функции-члены.

Составные операторы присвоения

Составные операторы присвоения не обязаны быть функциями-членами. Однако все операторы присвоения, включая составные, предпочтительно определять в классе. Для согласованности со встроенными составными операторами присвоения эти операторы должны возвращать ссылку на левый операнд. Например, ниже приведено определение составного оператора присвоения для класса `Sales_data`.

```
// бинарный оператор-член: левый операнд связан с неявным указателем this
// подразумевается, что оба объекта относятся к той же книге
Sales_data& Sales_data::operator+=(const Sales_data &rhs)
{
    units_sold += rhs.units_sold;
    revenue += rhs.revenue;
    return *this;
}
```



Обычно операторы присвоения и составные операторы присвоения должны быть определены как функции-члены и возвращать ссылку на левый операнд

Упражнения раздела 14.4

Упражнение 14.20. Определите оператор суммы и составной оператор присвоения для класса `Sales_data`.

Упражнение 14.21. Напишите операторы класса `Sales_data` так, чтобы `+` осуществлял сложение, а оператор `+=` вызывал оператор `+`. Обсудите недостатки этого подхода по сравнению со способом, которым эти операторы были определены в разделах 14.3 (стр. 710) и 14.4 (стр. 713).

Упражнение 14.22. Определите версию оператора присвоения, способного присвоить строку, представляющую ISBN, объекту класса `Sales_data`.

Упражнение 14.23. Определите в версии класса `StrVec` оператор присвоения для типа `initializer_list`.

Упражнение 14.24. Примите решение, нуждается ли класс из упражнения 7.40 раздела 7.5.1 (стр. 377) в операторах копирования и присваивания при перемещении. Если да, то определите эти операторы.

Упражнение 14.25. Реализуйте все остальные операторы присвоения, которые должен определить класс. Объясните, какие типы должны использоваться как операнды и почему.

14.5. Оператор индексирования

Классы, представляющие контейнеры, способные возвращать элементы по позиции, зачастую определяют оператор индексирования `operator[]`.



Оператор индексирования должен быть определен как функция-член класса.

Согласно общепринятому смыслу индексирования, оператор индексирования обычно возвращает ссылку на выбранный элемент. Возвращающий ссылку оператор индексирования применим с обеих сторон оператора присвоения. Следовательно, имеет смысл определить и константную, и неконстантную версии этого оператора. При применении к константному объекту оператор индексирования должен возвращать ссылку на константу, чтобы предотвратить присвоение возвращенному объекту.

★
Рекомендуем

Если у класса есть оператор индексирования, он обычно должен быть определен в двух версиях: возвращающей простую ссылку и являющуюся константной функцией-членом, а следовательно, возвращающую ссылку на константу.

В качестве примера определим оператор индексирования для класса `StrVec` (см. раздел 13.5, стр. 666):

```
class StrVec {
public:
    std::string& operator[](std::size_t n)
        { return elements[n]; }
    const std::string& operator[](std::size_t n) const
        { return elements[n]; }
    // другие члены, как в разделе 13.5 (стр. 666)
private:
    std::string *elements; // указатель на первый элемент массива
};
```

Эти операторы можно использовать таким же образом, как и индексирование вектора или массива. Поскольку оператор индексирования возвращает ссылку на элемент, если объект класса `StrVec` не константен, то этому элементу можно присвоить значение; если индексируется константный объект, присвоение невозможно:

```
// svec - объект класса StrVec
const StrVec cvec = svec; // копировать элементы из svec в cvec
// если у svec есть элементы, выполнить функцию empty() класса string
// для первого
if (svec.size() && svec[0].empty()) {
    svec[0] = "zero"; // ok: индексирование возвращает ссылку на строку
    cvec[0] = "Zip";  // ошибка: индексация cvec возвращает ссылку на
                      // константу
}
```

Упражнения раздела 14.5

Упражнение 14.26. Определите операторы индексирования для классов `StrVec`, `String`, `StrBlob` и `StrBlobPtr`.

14.6. Операторы инкремента и декремента

Операторы инкремента (`++`) и декремента (`--`) обычно реализуют для классов итераторов. Эти операторы позволяют перемещать итератор с элемента на элемент последовательности. Язык никак не требует, чтобы эти операторы были членами класса. Но поскольку они изменяют состояние объекта, с которым работают, лучше сделать их членами класса.

Для встроенных типов есть префиксные и постфиксные версии операторов инкремента и декремента. Ничего удивительного, что для собственных классов также можно определить префиксные и постфиксные версии этих операторов. Давайте сначала рассмотрим префиксные версии, а затем реализуем постфиксные.



Классы, определяющие операторы инкремента или декремента, должны определять как префиксные, так и постфиксные их версии. Обычно эти операторы определяют как функции-члены.

Определение префиксных версий операторов инкремента и декремента

Для иллюстрации операторов инкремента и декремента определим их для класса `StrBlobPtr` (см. раздел 12.1.6, стр. 604):

```
class StrBlobPtr {
public:
    // инкремент и декремент
    StrBlobPtr& operator++(); // префиксные операторы
    StrBlobPtr& operator--();
    // другие члены как прежде
};
```



Чтобы соответствовать встроенным типам, префиксные операторы должны возвращать ссылку на объект после инкремента или декремента.

Операторы инкремента и декремента работают подобным образом — они вызывают функцию `check()` для проверки допустимости объекта класса `StrBlobPtr`. Если это так, то функция `check()` проверяет также допустимость данного индекса. Если функция `check()` не передает исключения, эти операторы возвращают ссылку на свой объект.

В случае инкремента функции `check()` передается текущее значение `curr`. Пока это значение меньше размера основного вектора, функция `check()` завершается нормально. Если значение `curr` находится за концом вектора, функция `check()` передает исключение:

```

// префикс: возвращает ссылку на объект после инкремента
// или декремента
StrBlobPtr& StrBlobPtr::operator++()
{
    // если curr уже указывает после конца контейнера, инкремент
    // невозможен
    check(curr, "increment past end of StrBlobPtr");
    ++curr; // переместить текущую позицию вперед
    return *this;
}
StrBlobPtr& StrBlobPtr::operator--()
{
    // если curr равен нулю, то декремент возвратит недопустимый индекс
    --curr; // переместить текущую позицию назад
    check(-1, "decrement past begin of StrBlobPtr");
    return *this;
}

```

Оператор декремента уменьшает значение `curr` прежде, чем вызвать функцию `check()`. Таким образом, если значение `curr` (беззнаковое) уже является нулем, передаваемое функции `check()` значение будет наибольшим позитивным значением, представляющим недопустимый индекс (см. раздел 2.1.2, стр. 67).

Дифференциация префиксных и постфиксных операторов

При определении префиксных и постфиксных операторов возникает одна проблема: каждый из них имеет одинаковое имя и получает одинаковое количество параметров того же типа. При обычной перегрузке невозможно отличить префиксную и постфиксную версии оператора.

Для решения этой проблемы постфиксные версии получают дополнительный (неиспользуемый) параметр типа `int`. При использовании постфиксного оператора компилятор присваивает этому параметру аргумент 0. Хотя постфиксная функция вполне может использовать этот дополнительный параметр, как правило, так не поступают. Этот параметр не нужен для работы, обычно выполняемой постфиксным оператором. Его основная задача заключается в том, чтобы отличить определение постфиксной версии функции от префиксной.

Теперь в класс `CheckedPtr` можно добавить постфиксные операторы:

```

class StrBlobPtr {
public:
    // инкремент и декремент
    StrBlobPtr operator++(int); // постфиксные операторы
    StrBlobPtr operator--(int);
    // другие члены как прежде
};

```



Для совместимости со встроенными операторами постфиксные операторы должны возвращать прежнее значение (существовавшее до декремента или инкремента). Оно должно быть возвращено как значение, а не как ссылка.

Постфиксные версии должны запоминать текущее состояние объекта прежде, чем изменять объект:

```
,
// постфикс: инкремент/декремент объекта, но вернуть следует
// неизмененное значение
StrBlobPtr StrBlobPtr::operator++(int)
{
    // здесь проверка не нужна, ее выполнит префиксный инкремент
    StrBlobPtr ret = *this; // сохранить текущее значение
    ++*this;                // на один элемент вперед, проверку
                            // осуществляет оператор инкремента
    return ret;             // возврат сохраненного значения
}
StrBlobPtr StrBlobPtr::operator--(int)
{
    // здесь проверка не нужна, ее выполнит префиксный декремент
    StrBlobPtr ret = *this; // сохранить текущее значение
    --*this;                // на один элемент назад, проверку
                            // осуществляет оператор декремента
    return ret;             // возврат сохраненного значения
}
```

Для выполнения фактического действия каждый из этих операторов вызывает собственную префиксную версию. Например, постфиксный оператор инкремента использует такой вызов префиксного оператора инкремента:

```
++*this
```

Этот оператор проверяет безопасность приращения и либо передает исключение, либо осуществляет приращение значения `curr`. Если функция `check()` не передает исключения, постфиксные функции завершают работу, возвращая сохраненные ранее копии значений. Таким образом, после выхода сам объект будет изменен, но возвращено будет первоначальное, не измененное значение.



Поскольку параметр типа `int` не используется, имя ему присваивать не нужно.

Явный вызов постфиксных операторов

Как упоминалось на стр. 701, в качестве альтернативы использованию перегруженного оператора в выражении можно вызвать его явно. Если постфиксная версия задействуется при помощи вызова функции, то следует передать значение и для целочисленного аргумента:

```
StrBlobPtr p(a1); // p указывает на вектор в a1
p.operator++(0);  // вызов постфиксного оператора operator++
p.operator++();   // вызов префиксного оператора operator++
```

Переданное значение обычно игнорируется, но оно позволяет предупредить компилятор о том, что требуется именно постфиксная версия оператора.

Упражнения раздела 14.6

Упражнение 14.27. Добавьте в класс `StrBlobPtr` операторы инкремента и декремента.

Упражнение 14.28. Определите для класса `StrBlobPtr` операторы сложения и вычитания, чтобы они реализовали арифметические действия с указателями (см. раздел 3.5.3 стр. 169).

Упражнение 14.29. Почему не были определены константные версии операторов инкремента и декремента?

14.7. Операторы доступа к членам

Операторы обращения к значению (`*`) и стрелка (`->`) обычно используются в классах, представляющих итераторы, и в классах интеллектуального указателя (см. раздел 12.1, стр. 574). Вполне логично добавить эти операторы в класс `StrBlobPtr`:

```
class StrBlobPtr {
public:
    std::string& operator*() const
    { auto p = check(curr, "dereference past end");
      return (*p)[curr]; // (*p) - вектор, на который указывает этот
                        // объект
    }
    std::string* operator->() const
    { // передать реальную работу оператору обращения к значению
      return & this->operator*();
    }
    // другие члены как прежде
};
```

Оператор обращения к значению проверяет принадлежность `curr` диапазону, и если это так, то возвращает ссылку на элемент, обозначенный значением `curr`. Оператор стрелки не делает ничего сам, он вызывает оператор обращения к значению и возвращает адрес возвращенного им элемента.



Оператор стрелка (arrow) должен быть определен как функция-член класса. Оператор обращения к значению (dereference) необязательно должен быть членом класса, но, как правило, его тоже определяют как функцию-член.

Следует заметить, что эти операторы определены как константные члены. В отличие от операторов инкремента и декремента, выборка элемента никак не изменяет состояния объекта класса `StrBlobPtr`. Обратите также внимание на то, что эти операторы возвращают ссылку или указатель на неконстантную

строку. Причина этого в том, что объект класса `StrBlobPtr`, как известно, может быть связан только с неконстантным объектом класса `StrBlob` (см. раздел 12.1.6, стр. 604).

Эти операторы можно использовать таким же способом, которым используются соответствующие операторы с указателями и итераторами вектора:

```
StrBlob a1 = {"hi", "bye", "now"};
StrBlobPtr p(a1);           // p указывает на вектор в a1
*p = "okay";                // присвоить первый элемент a1
cout << p->size() << endl;   // выводит 4, размер первого элемента в a1
cout << (*p).size() << endl; // эквивалент p->size()
```

Ограничения на возвращаемое значение оператора стрелки

Подобно большинству других операторов (хотя это и плохая идея), оператор `operator*` можно определить как выполняющий некие действия по своему усмотрению. Таким образом, оператор `operator*` можно определить как возвращающий, например, фиксированное значение, скажем, 42, или выводящий содержимое объекта, к которому он применен, или что то еще. Но для перегруженного оператора стрелки это не так. Оператор стрелки никогда не изменяет своего фундаментального назначения: доступа к члену класса. При перегрузке оператора стрелки можно изменить объект, из которого стрелка выбирает определенный член, но нельзя изменить тот факт, что она выбирает член класса.

В коде `point->mem` часть `point` должна быть указателем на объект класса или объектом класса с перегруженным оператором `operator->`. В зависимости от типа части `point` код `point->mem` может быть эквивалентен следующему:

```
(*point).mem;           // point - указатель встроенного типа
point.operator()->mem;   // point - объект типа класса
```

В противном случае код ошибочен. Таким образом, код `point->mem` выполняется следующим образом.

1. Если `point` — указатель, то применение встроенного оператора стрелки означает эквивалент выражения `(*point).mem`. Указатель обращается к значению члена класса и выбирает его из объекта. Если у типа, на объект которого указывает `point`, нет члена по имени `mem`, то этот код ошибочен.
2. Если `point` — объект класса, в котором определен оператор `operator->`, то результат вызова `point.operator->()` используется для выбора члена `mem`. Если результат является указателем, то для него выполняется этап 1. Если результат является объектом, класс которого сам обладает перегруженным оператором `operator->()`, то с этим объектом повторяется данный этап. Процесс продолжается до тех пор, пока не будет возвращен указатель на объект с означенным членом или некое другое значение, означающее ошибочность кода.



Перегруженный оператор стрелки *должен* возвращать либо указатель на тип класса, либо объект типа класса, определяющего собственный оператор стрелки.

Упражнения раздела 14.7

Упражнение 14.30. Добавьте операторы обращения к значению и стрелки в класс `StrBlobPtr` и класс `ConstStrBlobPtr` из упражнения 12.22 раздела 12.1.6 (стр. 606). Обратите внимание, что операторы класса `ConstStrBlobPtr` должны возвращать константные ссылки, поскольку переменная-член `data` класса `ConstStrBlobPtr` указывает на константный вектор.

Упражнение 14.31. В классе `StrBlobPtr` не определен конструктор копий, оператор присвоения и деструктор. Почему?

Упражнение 14.32. Определите класс, содержащий указатель на класс `StrBlobPtr`. Определите перегруженный оператор стрелки для этого класса.



14.8. Оператор вызова функции

Классы, перегружающие оператор вызова, позволяют использовать объекты этого типа как функции. Поскольку объекты таких классов способны хранить состояние, они могут оказаться существенно гибче обычных функций.

В качестве простого примера рассмотрим структуру `absInt`, обладающую оператором вызова, возвращающим абсолютное значение своего аргумента:

```
struct absInt {
    int operator()(int val) const {
        return val < 0 ? -val : val;
    }
};
```

Этот класс определяет одну функцию: оператор вызова функции. Этот оператор получает аргумент типа `int` и возвращает абсолютное значение аргумента.

Оператор вызова используется применительно к списку аргументов объекта класса `absInt` способом, который выглядит как вызов функции:

```
int i = -42;
absInt absObj;           // объект класса с оператором вызова функции
int ui = absObj(i);       // передача i в absObj.operator()
```

Хотя `absObj` — это объект, а не функция, его вполне можно вызвать. При вызове объект выполняет свой перегруженный оператор вызова. В данном случае этот оператор получает значение типа `int` и возвращает его абсолютное значение.



Оператор вызова функции должен быть функцией-членом. Класс может определить несколько версий оператора вызова, каждая из которых должна отличаться количеством или типом параметров.

Объект класса, определяющего оператор вызова, называется *объектом функции* (function object). Такие объекты действуют как функции, поскольку их можно вызвать.

Классы объектов функций с состоянием

У класса объекта функции, как у любого другого класса, могут быть и другие члены, кроме оператора `operator()`. Классы объекта функции зачастую содержат переменные-члены, используемые для настройки действий в операторе вызова.

В качестве примера определим класс, выводящий строковый аргумент. По умолчанию класс будет писать в поток `cout` и выводить пробел после каждой строки. Позволим также пользователям класса предоставлять другой поток для записи и другой разделитель. Этот класс можно определить следующим образом:

```
class PrintString {
public:
    PrintString(ostream &o = cout, char c = ' '):
        os(o), sep(c) { }
    void operator()(const string &s) const { os << s << sep; }
private:
    ostream &os; // поток для записи
    char sep;    // символ завершения после каждого вывода
};
```

У класса есть конструктор, получающий ссылку на поток вывода, и символ, используемый как разделитель. Как аргументы по умолчанию (см. раздел 6.5.1, стр. 308) для этих параметров используется поток `cout` и пробел. Тело оператора вызова функции использует эти члены при выводе данной строки.

При определении объектов класса `PrintString` можно использовать аргументы по умолчанию или предоставлять собственные значения для разделителя или потока вывода:

```
PrintString printer; // использует аргументы по умолчанию; вывод в cout
printer(s);          // выводит s и пробел в cout
PrintString errors(cerr, '\n');
errors(s);           // выводит s и новую строку в cerr
```

Объекты функции обычно используют как аргументы для обобщенных алгоритмов. Например, для вывода содержимого контейнера можно использовать класс `PrintString` и библиотечный алгоритм `for_each()` (см. раздел 10.3.2, стр. 500):

```
for_each(vs.begin(), vs.end(), PrintString(cerr, '\n'));
```

Третий аргумент алгоритма `for_each()` является временным объектом типа `PrintString`, инициализируемый потоком `cerr` и символом новой строки. Вызов функции `for_each()` выводит каждый элемент `vs` в поток `cerr`, разделяя их новой строкой.

Упражнения раздела 14.8

Упражнение 14.33. Сколько операндов может иметь перегруженный оператор вызова функции?

Упражнение 14.34. Определите класс объекта функции для выполнения действий условного оператора: оператор вызова этого класса должен получать три параметра. Он должен проверить свой первый параметр и, если эта проверка успешна, вернуть свой второй параметр; в противном случае он должен вернуть свой третий параметр.

Упражнение 14.35. Напишите класс, подобный классу `PrintString`, который читает строку из потока `istream` и возвращает строку, представляющую прочитанное. При неудаче чтения следует вернуть пустую строку.

Упражнение 14.36. Используя класс из предыдущего упражнения, организуйте чтение со стандартного устройства ввода, сохраняя каждую строку в векторе как элемент.

Упражнение 14.37. Напишите класс, проверяющий равенство двух значений. Используйте этот объект и библиотечные алгоритмы для написания кода замены всех экземпляров заданного значения в последовательности.

14.8.1. Лямбда-выражения — объекты функции

В предыдущем разделе объект `PrintString` использовался как аргумент в вызове функции `for_each()`. Это похоже на программу, написанную в разделе 10.3.2 (стр. 497), где использовалось лямбда-выражение. Написанное лямбда-выражение компилятор преобразовывает в безымянный объект безымянного класса (см. раздел 10.3.3, стр. 502). Классы, созданные из лямбда-выражения, содержат перегруженный оператор вызова функции. Рассмотрим, например, лямбда-выражение, передававшееся как последний аргумент функции `stable_sort()`:

```
// сортировать слова по размеру, поддерживая алфавитный порядок среди
// слов того же размера
stable_sort(words.begin(), words.end(),
            [](const string &a, const string &b)
            { return a.size() < b.size();});
```

Это действует как безымянный объект класса, который выглядел бы примерно так:

```
class ShorterString {
public:
    bool operator()(const string &s1, const string &s2) const
    { return s1.size() < s2.size(); }
};
```

У этого класса есть один член, являющийся оператором вызова функции, получающим две строки и сравнивающий их длины. Список параметров и тело функции те же, что и у лямбда-выражения. Как уже упоминалось в разделе 10.3.3 (стр. 506), по умолчанию лямбда-выражения не могут изменять свои захваченные переменные. В результате по умолчанию оператор вызова функции в классе, созданном из лямбда-выражения, является константной функцией-членом. Если лямбда-выражение объявляется как `mutable`, то оператор вызова не будет константным.

Вызов функции `stable_sort()` можно переписать так, чтобы использовать этот класс вместо лямбда-выражения:

```
stable_sort(words.begin(), words.end(), ShorterString());
```

Третий аргумент — недавно созданный составной объект класса `ShorterString`. Код в функции `stable_sort()` будет вызывать этот объект каждый раз, когда он сравнивает две строки. При вызове объекта будет выполнено тело его оператора вызова, возвращающего значение `true`, если размер первой строки будет меньше, чем второй.

Классы, представляющие лямбда-выражения с захваченными переменными

Как уже упоминалось, при захвате лямбда-выражением переменной по ссылке разработчик должен сам гарантировать существование переменной, на которую ссылается ссылка, во время выполнения лямбда-выражения (см. раздел 10.3.3, стр. 502). Поэтому компилятору разрешено использовать ссылку непосредственно, не сохраняя ее как переменную-член в созданном классе.

Переменные, которые захватываются по значению, напротив, копируются в лямбда-выражение (см. раздел 10.3.3, стр. 502). В результате классы, созданные из лямбда-выражений, переменные которых захватываются по значению, имеют переменные-члены, соответствующие каждой такой переменной. У этих классов есть также конструктор для инициализации этих переменных-членов значениями захваченных переменных. В примере раздела 10.3.2 (стр. 500) лямбда-выражение использовалось для поиска первой строки, длина которой была больше или равна заданному значению:

```
// получить итератор на первый элемент, размер которого >= sz
auto wc = find_if(words.begin(), words.end(),
    [sz](const string &a)
```

Созданный класс выглядел бы примерно так:

```
class SizeComp {
    SizeComp(size_t n): sz(n) { } // параметр для каждой захваченной
                                // переменной
    // оператор вызова с тем же типом возвращаемого значения, параметрами
    // и телом, как у лямбда-выражения
    bool operator()(const string &s) const
    { return s.size() >= sz; }
private:
    size_t sz; // переменная-член для каждой переменной, захваченной
              // по значению
};
```

В отличие от класса `ShorterString`, у этого класса есть переменная-член и конструктор для ее инициализации. У этого синтезируемого класса нет стандартного конструктора; чтобы использовать этот класс, следует передать аргумент:

```
// получить итератор на первый элемент, размер которого >= sz
auto wc = find_if(words.begin(), words.end(), SizeComp(sz));
```

У классов, созданных из лямбда-выражения, есть удаленный стандартный конструктор, удаленные операторы присвоения и стандартный деструктор. Будет ли у класса стандартный или удаленный конструктор копий/перемещения, зависит обычно от способа и типа захватываемых переменных-членов (см. раздел 13.1.6, стр. 644 и раздел 13.6.2, стр. 680).

Упражнения раздела 14.8.1

Упражнение 14.38. Напишите класс, проверяющий соответствие длины заданной строки указанному значению. Используйте такой объект в программе для оповещения о количестве слов во входном файле, имеющих размеры от 1 до 10 включительно.

Упражнение 14.39. Перепишите предыдущую программу так, чтобы сообщать количество слов размером от 1 до 9 и 10 или более.

Упражнение 14.40. Перепишите функцию `biggies()` из раздела 10.3.2 (стр. 500) так, чтобы использовать объект функции вместо лямбда-выражения.

Упражнение 14.41. Как по-вашему, существенно ли добавление лямбда-выражений по новому стандарту? Объясните, когда имеет смысл использовать лямбда-выражение, а когда класс вместо него.

14.8.2. Библиотечные объекты функций

Стандартная библиотека определяет набор классов, представляющих арифметические, реляционные и логические операторы. Каждый класс опре-

деляет оператор вызова, который применяет одноименный оператор. Например, у класса `plus` есть оператор вызова функции, который применяет оператор `+` к паре операндов; класс `modulus` определяет оператор вызова, применяющий бинарный оператор `%`; класс `equal_to` применяет оператор `==`; и т.д.

Эти классы являются шаблонами, которым передается один тип. Он определяет тип параметра оператора вызова. Например, класс `plus<string>` применяет строковый оператор суммы к объектам класса `string`; у класса `plus<int>` типом операндов будет `int`; класс `plus<Sales_data>` применяет оператор `+` к объектам класса `Sales_data`; и т.д.:

```
plus<int> intAdd;           // объект функции, способный сложить
                           // два значения типа int
negate<int> intNegate;     // объект функции, способный изменить знак
                           // значения типа int
// использование оператора intAdd::operator(int, int) для
// сложения чисел 10 и 20
int sum = intAdd(10, 20);   // эквивалент sum = 30
sum = intNegate(intAdd(10, 20)); // эквивалент tosum = 30
// использование оператора intNegate::operator(int) для создания
// числа -10 как второго параметра выражения intAdd::operator(int, int)
sum = intAdd(10, intNegate(10)); // sum = 0
```

Эти типы, перечислены в табл. 4.2, определены в заголовке `functional`.

Таблица 14.2. Библиотечные объекты функций

Арифметические	Реляционные	Логические
<code>plus<Type></code>	<code>equal_to<Type></code>	<code>logical_and<Type></code>
<code>minus<Type></code>	<code>not_equal_to<Type></code>	<code>logical_or<Type></code>
<code>multiplies<Type></code>	<code>greater<Type></code>	<code>logical_not<Type></code>
<code>divides<Type></code>	<code>greater_equal<Type></code>	
<code>modulus<Type></code>	<code>less<Type></code>	
<code>negate<Type></code>	<code>less_equal<Type></code>	

Применение библиотечного объекта функции с алгоритмами

Классы объектов функций, представляющие операторы, зачастую используются для переопределения заданного по умолчанию оператора, используемого алгоритмом. Как уже упоминалось, по умолчанию алгоритмы сортировки используют оператор `operator<` для сортировки последовательности в порядке возрастания. Для сортировки в порядке убывания можно передать объект типа `greater`. Этот класс создает оператор вызова, который вызывает оператор “больше” основного типа элемента. Предположим, например, что `svec` — это вектор типа `vector<string>`:

```
// передает временный объект функции, который применяет
// оператор > к двум строкам
sort(svec.begin(), svec.end(), greater<string>());
```

Это сортирует вектор в порядке убывания. Третий аргумент — безымянный объект типа `greater<string>`. Когда функция `sort()` сравнит элементы, вместо оператора `<` типа элемента она применит переданный объект функции `greater`. Этот объект применит оператор `>` к элементам типа `string`.

Одним из важнейших аспектов этих библиотечных объектов функций является то, что библиотека гарантирует их работоспособность с указателями. Помните, что результат сравнения двух несвязанных указателей непредсказуем (см. раздел 3.5.3, стр. 171). Но может понадобиться сортировать вектор указателей на основании их адреса в памяти. Хотя сделать это самостоятельно не просто, вполне можно применить один из библиотечных объектов функции:

```
vector<string *> nameTable; // вектор указателей
// ошибка: указатели в nameTable не связаны, результат < непредсказуем
sort(nameTable.begin(), nameTable.end(),
      [](string *a, string *b) { return a < b; });
// ok: библиотека гарантирует, что less для типов указателя определен
sort(nameTable.begin(), nameTable.end(), less<string*>());
```

Стоит также обратить внимание на то, что ассоциативные контейнеры используют для упорядочивания своих элементов объект типа `less<key_type>`. В результате можно определить набор (`set`) указателей или использовать указатель как ключ в карте (`map`) без необходимости определять тип `less` самостоятельно.

Упражнения раздела 14.8.2

Упражнение 14.42. Используя библиотечные объекты и адаптеры функций, определите объекты для:

- (a) Подсчета количеств значений больше 1024
- (b) Поиска первой строки, не равной `pooh`
- (c) Умножения всех значений на 2

Упражнение 14.43. Используя библиотечные объекты функций, определите, делимо ли переданное значение типа `int` на некий элемент в контейнере целых чисел.

14.8.3. Вызываемые объекты и тип `function`

В языке C++ есть несколько видов вызываемых объектов: функции и указатели на функции, лямбда-выражения (см. раздел 10.3.2, стр. 497), объекты, созданные функцией `bind()` (см. раздел 10.3.4, стр. 508), и классы с перегруженным оператором вызова функции.

Подобно любому другому объекту, у вызываемого объекта есть тип. Например, у каждого лямбда-выражения есть собственный уникальный (безымянный) тип класса. Типы функций и указателей на функции зависят от типа возвращаемого значения, типа аргумента и т.д.

Однако два вызываемых объекта с разными типами могут иметь ту же *сигнатуру вызова* (call signature). Сигнатура вызова определяет тип возвращаемого значения вызываемого объекта и тип (типы) аргумента, которые следует передать при вызове. Сигнатура вызова соответствует типу функции. Например:

```
int(int, int)
```

Функция этого типа получает два числа типа `int` и возвращает значение типа `int`.

Разные типы могут иметь одинаковую сигнатуру вызова

Иногда необходимо использовать несколько вызываемых объектов с одинаковой сигнатурой вызова, как будто это тот же тип. Рассмотрим, например, следующие разные типы вызываемых объектов:

```
// обычная функция
int add(int i, int j) { return i + j; }
// лямбда-выражение, создающее безымянный класс объекта функции
auto mod = [](int i, int j) { return i % j; };
// класс объекта функции
struct div {
    int operator()(int denominator, int divisor) {
        return denominator / divisor;
    }
};
```

Каждый из этих вызываемых объектов применяет арифметическую операцию к своим параметрам. Даже при том, что у каждого из них разный тип, сигнатура вызова у них одинакова:

```
int(int, int)
```

Эти вызываемые объекты можно использовать для написания простого калькулятора. Для этого следует определить *таблицу функций* (function table), хранящую “указатели” на вызываемые объекты. Когда программе понадобится выполнить некую операцию, она просмотрит таблицу и найдет соответствующую функцию.

В языке C++ таблицы функций довольно просто реализовать при помощи карт (map). В данном случае как ключ используем строку, соответствующую символу оператора; значение будет функцией, реализующей этот оператор. При необходимости выполнить заданный оператор индексируется карта и осуществляется вызов возвращенного элемента. Если бы все эти функции были автономными и необходимо было использовать только парные операторы для типа `int`, то карту можно было бы определить так:

```
// сопоставляет оператор с указателем на функцию, получающую два целых
// числа и возвращающую целое число
map<string, int(*) (int, int)> binops;
```

Указатель `add` можно поместить в карту `binops` следующим образом:

```
// ok: add - указатель на функцию соответствующего типа
binops.insert({"+", add}); // {"+", add} - пара раздел 11.2.3 (стр. 545)
```

Но сохранить в карте `binops` объекты `mod` или `div` не получится:

```
binops.insert({"%", mod}); // ошибка: mod - не указатель на функцию
```

Проблема в том, что `mod` — это лямбда-выражение, и у каждого лямбда-выражения есть собственный тип класса. Этот тип не соответствует типу значений, хранимых в карте `binops`.

Библиотечный тип `function`

C++
11

Эту проблему можно решить при помощи нового библиотечного типа `function`, определенного в заголовке `functional`; возможные операции с типом `function` приведены в табл. 14.3.

Таблица 14.3. Операции с типом `function`

<code>function<T> f;</code>	<code>f</code> — пустой объект класса <code>function</code> , способный хранить вызываемые объекты с сигнатурой вызова, эквивалентной типу функции <code>T</code> (т.е. <code>T</code> — это <code>retType(args)</code>)
<code>function<T> f(nullptr);</code>	Явное создание пустого объекта класса <code>function</code>
<code>function<T> f(obj);</code>	Сохранение копии вызываемого объекта <code>obj</code> в объекте <code>f</code>
<code>f</code>	Когда <code>f</code> используется как условие; оно истинно, если содержит вызываемый объект, и ложно в противном случае
<code>f(args)</code>	Вызывает объект <code>f</code> с передачей аргументов <code>args</code>
Типы, определенные как члены шаблона <code>function<T></code>	
<code>result_type</code>	Тип возвращаемого значения объекта функции этого типа
<code>argument_type</code>	Типы, определяемые, когда у типа <code>T</code> есть один или два аргумента. Если у типа <code>T</code> есть один аргумент, то
<code>first_argument_type</code>	<code>argument_type</code> — синоним его типа. Если у типа <code>T</code> два аргумента, то <code>first_argument_type</code> и
<code>second_argument_type</code>	<code>second_argument_type</code> — синонимы их типов

Тип `function` — это шаблон. Подобно другим шаблонам, при создании его экземпляра следует указать дополнительную информацию. В данном случае этой информацией является сигнатура вызова объекта, который сможет представлять данный конкретный тип `function`. Как и у других шаблонов, этот тип определяют в угловых скобках:

```
function<int(int, int)>
```

Здесь был объявлен тип `function`, способный представлять вызываемые объекты, возвращающие целочисленный результат и имеющие два параметра типа `int`. Этот тип можно использовать для представления любого из типов приложения калькулятора:

```
function<int(int, int)> f1 = add;    // указатель на функцию
function<int(int, int)> f2 = div(); // объект класса объекта функции
function<int(int, int)> f3 = [](int i, int j) // лямбда-выражение
    { return i * j; };
cout << f1(4,2) << endl; // выводит 6
cout << f2(4,2) << endl; // выводит 2
cout << f3(4,2) << endl; // выводит 8
```

Теперь карту можно переопределить, используя тип `function`:

```
// таблица вызываемых объектов, соответствующих всем бинарным операторам
// все вызываемые объекты должны получать по два int и возвращать int
// элемент может быть указателем на функцию, объектом функции или
// лямбда-выражением
map<string, function<int(int, int)>> binops;
```

В эту карту можно добавить каждый из вызываемых объектов приложения, будь то указатель на функцию, лямбда-выражение или объект функции:

```
map<string, function<int(int, int)>> binops = {
    {"+", add},                // указатель на функцию
    {"-", std::minus<int>()},  // объект библиотечной функции
    {"/", div()},              // пользовательский объект функции
    {"*", [](int i, int j) { return i * j; }}, // безымянное
                                // лямбда-выражение
    {"%", mod} };              // именованный объект лямбда-выражения
```

В карте пять элементов. Хотя все лежащие в основе вызываемые объекты имеют различные типы, каждый из них можно хранить в общем типе `function<int(int, int)>`.

Как обычно, при индексировании карты возвращается ссылка на ассоциированное значение. При индексировании карты `binops` возвращается ссылка на объект типа `function`. Тип `function` перегружает оператор вызова. Этот оператор вызова получает собственные аргументы и передает их хранимому вызываемому объекту:

```
binops["+"](10, 5); // вызов add(10, 5)
binops["-"](10, 5); // использует оператор; вызов объекта minus<int>
binops["/"](10, 5); // использует оператор; вызов объекта div
binops["*"](10, 5); // вызов объекта лямбда-функции
binops["%"](10, 5); // вызов объекта лямбда-функции
```

Здесь происходит вызов каждой из операций, хранимых в карте `binops`. В первом вызове возвращаемый элемент является указателем на функцию, указывающим на функцию `add()`. Вызов `binops["+"](10, 5)` использует этот указатель для вызова функции `add` с передачей ей значений 10 и 5. Сле-

дующий вызов, `binops["-"]`, возвращает объект класса `function`, хранящий объект типа `std::minus<int>`. Затем можно вызвать оператор этого объекта и других.

Перегруженные функции и тип `function`

Нельзя непосредственно хранить имя перегруженной функции в объекте типа `function`:

```
int add(int i, int j) { return i + j; }
Sales_data add(const Sales_data&, const Sales_data&);
map<string, function<int(int, int)>> binops;
binops.insert( {"+", add} ); // ошибка: какой именно add?
```

Один из способов разрешения двусмысленности подразумевает хранение указателя на функцию (см. раздел 6.7, стр. 322) вместо имени функции:

```
int (*fp)(int, int) = add; // указатель на версию add, получающую два int
binops.insert( {"+", fp} ); // ok: fp указывает на правую версию add
```

В качестве альтернативы для устранения неоднозначности можно использовать лямбда-выражение:

```
// ok: использование лямбда-выражения для устранения неоднозначности при
// выборе используемой версии add
binops.insert( {"+", [](int a, int b) {return add(a, b);} } );
```

Вызов в теле лямбда-выражения передает два целых числа. Этому вызову может соответствовать только та версия функции `add()`, которая получает два целых числа, а следовательно, эта функция и применяется при выполнении лямбда-выражения.



Класс `function` в новой библиотеке никак не связан с классами `unary_function` и `binary_function`, которые были частью прежних версий библиотеки. Эти классы были заменены более общей функцией `bind()` (см. раздел 10.3.4, стр. 513).

Упражнения раздела 14.8.3

Упражнение 14.44. Напишите собственную версию простого калькулятора, способного выполнять бинарные операции.



14.9. Перегрузка, преобразование и операторы

В разделе 7.5.4 (стр. 380) упоминалось, что неявный конструктор, который может быть вызван с одним аргументом, определяет неявное преобразование. Такие конструкторы преобразовывают объект типа аргумента в тип класса.

Можно также определить преобразование из типа класса. Для этого нужно определить оператор преобразования. Конструкторы преобразования и операторы преобразования определяют *преобразования типа класса* (class-type conversion). Такие преобразования называются также *пользовательскими преобразованиями* (user-defined conversion).

14.9.1. Операторы преобразования

Оператор преобразования (conversion operator) — это специальный вид функции-члена класса. Общий синтаксис функции преобразования имеет следующий вид:

```
operator тип() const;
```

где *тип* — это имя типа. Операторы преобразования могут быть определены для любого типа (кроме `void`), который может быть типом возвращаемого значения функции (см. раздел 6.1, стр. 270). Преобразование в тип массива или функции недопустимо. Однако преобразование в тип указателя на данные или функцию, а также ссылочные типы вполне возможны.

У операторов преобразования нет явно заданного типа возвращаемого значения и нет параметров, их следует определять как функции-члены. Операции преобразования обычно не должны изменять преобразуемый объект. В результате операторы преобразования обычно определяют как константные члены.



Функция преобразования должна быть функцией-членом, у нее не определен тип возвращаемого значения и пустой список параметров. Функция обычно должна быть константой.

Определение класса с оператором преобразования

Для примера определим небольшой класс, представляющий целое число в диапазоне от 0 до 255:

```
class SmallInt {
public:
    SmallInt(int i = 0): val(i)
    {
        if (i < 0 || i > 255)
            throw std::out_of_range("Bad SmallInt value");
    }
    operator int() const { return val; }
private:
    std::size_t val;
};
```

Класс `SmallInt` определяет преобразования *в* и *из* своего типа. Конструктор преобразует значения арифметического типа в тип `SmallInt`. Оператор преобразования преобразует объекты класса `SmallInt` в тип `int`:

```

SmallInt si;
si = 4; // неявно преобразует 4 в SmallInt, а затем
        // вызывает SmallInt::operator=
si + 3; // неявно преобразует si в int с последующим целочисленным
        // суммированием

```

Хотя компилятор применяет только одно пользовательское преобразование за раз (см. раздел 4.11.2, стр. 222), неявное пользовательское преобразование можно предварить или сопроводить стандартным (встроенным) преобразованием (см. раздел 4.11.1, стр. 219). В результате конструктору `SmallInt` можно передать любой арифметический тип. Точно так же можно использовать оператор преобразования для преобразования объекта класса `SmallInt` в `int`, а затем преобразовать полученное значение типа `int` в другой арифметический тип:

```

// аргумент типа double преобразуется в int с использованием встроенного
// преобразования
SmallInt si = 3.14; // вызов конструктора SmallInt(int)
// оператор преобразования класса SmallInt преобразует si в int
si + 3.14; // int преобразуется в double с использованием встроенного
           // преобразования

```

Поскольку операторы преобразования применяются неявно, нет никакого способа передать аргументы этим функциям. Следовательно, операторы преобразования не могут быть определены как получающие параметры. Хотя функция преобразования не определяет тип возвращаемого значения, каждая из них должна вернуть значение соответствующего типа:

```

class SmallInt;
operator int(SmallInt&);           // ошибка: не член класса
class SmallInt {
public:
    int operator int() const;       // ошибка: тип возвращаемого значения
    operator int(int = 0) const;    // ошибка: список параметров
    operator int*() const { return 42; } // ошибка: 42 не указатель
};

```

ВНИМАНИЕ! НЕ ЗЛОУПОТРЕБАЙТЕ ФУНКЦИЯМИ ПРЕОБРАЗОВАНИЯ

Как и в случае с перегруженными операторами, разумное использование функций преобразования помогает существенно упростить работу разработчика класса и сделать полученный класс удобным в применении. Однако здесь есть две потенциальные ловушки: определение слишком большого количества функций преобразования может привести к неоднозначности кода, а некоторые преобразования могут оказаться скорее вредными, чем полезными.

Для примера рассмотрим класс `Date`, представляющий данные о дате. Вполне очевидно, что имеет смысл предоставить способ преобразования

объекта класса `Date` в объект типа `int`. Но какое значение должна возвращать функция преобразования? Она могла бы вернуть десятичное представление года, месяца и дня. Например, 30 июля 1989 года могло бы быть представлено как значение 19800730 типа `int`. В качестве альтернативы оператор преобразования мог бы возвращать целое число, соответствующее количеству дней, начиная с некоторой эпохальной даты. Счетчик мог бы считать дни с 1 января 1970 года или некой другой отправной точки. У обоих преобразований есть желаемое свойство, что более поздние даты соответствуют большим целым числам, что может быть очень полезно.

Проблема в том, что нет единого и полного соответствия между объектом типа `Date` и значением типа `int`. В таких случаях лучше не определять оператор преобразования. Вместо него класс должен определить один или несколько обычных членов, чтобы извлекать эту информацию в различных форматах.

Операторы преобразования могут привести к удивительным результатам

На практике классы редко предоставляют операторы преобразования. Пользователи, вероятней всего, будут просто удивлены, случись преобразование автоматически, без помощи явного преобразования. Но из этого эмпирического правила есть одно важное исключение: преобразование в тип `bool` является вполне общепринятым для классов.

По прежним версиям стандарта перед классами с преобразованием в тип `bool` стояла проблема: поскольку тип `bool` арифметический, объект этого типа, допускающего преобразование в тип `bool`, применим в любом контексте, где ожидается арифметический тип. Такие преобразования могут происходить весьма удивительными способами. В частности, если бы у класса `istream` было преобразование в тип `bool`, то следующий код вполне компилировался бы:

```
int i = 42;
cin << i; // этот код был бы допустим, если бы преобразование в тип bool
         // не было явным!
```

Эта программа пытается использовать оператор вывода для входного потока. Для класса `istream` оператор `<<` не определен, поэтому такой код безусловно ошибочен. Но этот код мог бы использовать оператор преобразования в тип `bool`, чтобы преобразовать объект `cin` в `bool`. Полученное значение типа `bool` было бы затем преобразовано в тип `int`, который вполне применим как левый операнд встроенной версии оператора сдвига влево. В результате преобразованное значение типа `bool` (1 или 0) было бы сдвинуто влево на 42 позиции.



Явный оператор преобразования

Чтобы предотвратить подобные проблемы, новый стандарт вводит *явный оператор преобразования* (explicit conversion operator):

```
class SmallInt {
public:
    // компилятор не будет автоматически применять это преобразование
    explicit operator int() const { return val; }
    // другие члены как прежде
};
```

Подобно явным конструкторам (см. раздел 7.5.4, стр. 383), компилятор не будет (обычно) использовать явный оператор преобразования для неявных преобразований:

```
SmallInt si = 3; // ok: конструктор класса SmallInt не является явным
si + 3; // ошибка: нужно неявное преобразование, но оператор int
        // является явным
static_cast<int>(si) + 3; // ok: явный запрос преобразования
```

Если оператор преобразования является явным, такое преобразование вполне можно осуществить. Но за одним исключением такое приведение следует осуществить явно.

Исключение состоит в том, что компилятор применит явное преобразование в выражении, используемом как условие. Таким образом, явное преобразование будет использовано неявно для преобразования выражения, используемого как:

- условие оператора if, while или do;
- выражение условия в заголовке оператора for;
- операнд логического оператора NOT (!), OR (||) или AND (&&);
- выражение условия в условном операторе (? :).

Преобразование в тип bool

В прежних версиях библиотеки типы ввода-вывода определяли преобразование в тип void*. Это было сделано во избежание проблем, описанных выше. По новому стандарту библиотека ввода-вывода определяет вместо этого явное преобразование в тип bool.

Всякий раз, когда потоковый объект используется в условии, применяется оператор `operator bool()`, определенный для типов ввода-вывода. Например:

```
while (std::cin >> value)
```

Условие в операторе while выполняет оператор ввода, который читает в переменную value и возвращает объект cin. Для обработки условия объект cin неявно преобразуется функцией преобразования `istream operator bool()`. Эта функция возвращает значение true, если флагом состояния потока cin является good (см. раздел 8.1.2, стр. 402), и false в противном случае.



Преобразование в тип `bool` обычно используется в условиях. В результате оператор `operator bool` обычно должен определяться как явный.

Упражнения раздела 14.9.1

Упражнение 14.45. Напишите операторы преобразования для преобразования объекта класса `Sales_data` в значения типа `string` и `double`. Какие значения, по-вашему, должны возвращать эти операторы?

Упражнение 14.46. Объясните, является ли определение этих операторов преобразования класса `Sales_data` хорошей идеей и должны ли они быть явными.

Упражнение 14.47. Объясните различие между этими двумя операторами преобразования:

```
struct Integral {  
    operator const int();  
    operator int() const;  
};
```

Упражнение 14.48. Должен ли класс из упражнения 7.40 раздела 7.5.1 (стр. 377) использовать преобразование в тип `bool`. Если да, то объясните почему и укажите, должен ли оператор быть явным. В противном случае объясните, почему нет.

Упражнение 14.49. Независимо от того, хороша ли эта идея, определите преобразование в тип `bool` для класса из предыдущего упражнения.



14.9.2. Избегайте неоднозначных преобразований

Если у класса есть один или несколько операторов преобразования, важно гарантировать наличие только одного способа преобразования из типа класса в необходимый тип. Если будет больше одного способа осуществления преобразования, то будет весьма затруднительно написать однозначный код.

Есть два случая, когда возникает несколько путей осуществления преобразования. Первый — когда два класса обеспечивают взаимное преобразование. Например, взаимное преобразование осуществляется тогда, когда класс `A` определяет конструктор преобразования, получающий объект класса `B`, а класс `B` определяет оператор преобразования в тип `A`.

Второй случай возникновения нескольких путей преобразования — определение нескольких преобразований в и из типов, которые сами связаны преобразованиями. Самый очевидный пример — встроенные арифметические типы. Каждый класс обычно должен определять не больше одного преобразования в или из арифметического типа.



Обычно не следует определять классы со взаимными преобразованиями или определять преобразования в или из арифметических типов.

Распознавание аргумента и взаимные преобразования

В следующем примере определены два способа получения объекта класса A из B: либо при помощи оператора преобразования класса B, либо при помощи конструктора класса A, получающего объект класса B:

```
// обычно взаимное преобразование между двумя типами - плохая идея
struct B;
struct A {
    A() = default;
    A(const B&); // преобразует B в A
    // другие члены
};
struct B {
    operator A() const; // тоже преобразует B в A
    // другие члены
};
A f(const A&);
B b;
A a = f(b); // ошибка неоднозначности: f(B::operator A())
            // или f(A::A(const B&))
```

Поскольку существуют два способа получения объекта класса A из B, компилятор не знает, какой из них использовать; поэтому вызов функции `f()` неоднозначен. Для получения объекта класса B этот вызов может использовать конструктор класса A или оператор преобразования класса B, преобразующий объект класса B в A. Поскольку обе эти функции одинаково хороши, вызов неоднозначен и ошибочен.

Если этот вызов необходим, оператор преобразования или конструктор следует вызвать явно:

```
A a1 = f(b.operator A()); // ok: использовать оператор преобразования B
A a2 = f(A(b));           // ok: использовать конструктор класса A
```

Обратите внимание: нельзя решить неоднозначность при помощи приведения — у самого приведения будет та же двусмысленность.

Двусмысленность и множественность путей преобразования во встроенные типы

Двусмысленность возникает также в случае, когда класс определяет несколько преобразований в (или из) типы, которые сами связываются преобразованиями. Самый простой и наглядный пример (а также особенно проблематичный) — это когда класс определяет конструкторы преобразования в или из более, чем один арифметический тип.

Например, у следующего класса есть конструкторы преобразования из двух разных арифметических типов и операторы преобразования в два разных арифметических типа:

```
struct A {
    A(int = 0);           // обычно плохая идея иметь два
    A(double);           // преобразования из арифметических типов
    operator int() const; // обычно плохая идея иметь два
    operator double() const; // преобразования в арифметические типы
    // другие члены
};
void f2(long double);
A a;
f2(a); // ошибка неоднозначности: f(A::operator int())
      // или f(A::operator double())
long lg;
A a2(lg); // ошибка неоднозначности: A::A(int) или A::A(double)
```

В вызове функции `f2()` ни одно из преобразований не соответствует точно типу `long double`. Но для его получения применимо любое преобразование, сопровождаемое стандартным преобразованием. Следовательно, никакое из преобразований не лучше другого, значит, вызов неоднозначен.

Возникает та же проблема, что и при попытке инициализации объекта `a2` значением типа `long`. Ни один из конструкторов не соответствует точно типу `long`. Каждый требовал преобразования аргумента прежде, чем использовать конструктор.

- Стандартное преобразование `long` в `double`, затем `A(double)`.
- Стандартное преобразование `long` в `int`, затем `A(int)`.

Эти последовательности преобразований равнозначны, поэтому вызов неоднозначен.

Вызов функции `f2()` и инициализация объекта `a2` неоднозначны, поскольку у необходимых стандартных преобразований одинаковый ранг (см. раздел 6.6.1, стр. 320). Когда используется пользовательское преобразование, ранг стандартного преобразования, если таковые вообще имеются, позволяет выбрать наилучшее соответствие:

```
short s = 42;
// преобразование short в int лучше, чем short в double
A a3(s); // используется A::A(int)
```

В данном случае преобразование `short` в `int` предпочтительней, чем `short` в `double`. Следовательно, объект `a3` создается с использованием конструктора `A::A(int)`, который запускается для преобразования значения `s`.



ВНИМАНИЕ

Когда используются два пользовательских преобразования, ранг стандартного преобразования, если таковое вообще имеется, используется для выбора наилучшего соответствия.

Перегруженные функции и конструкторы преобразования

Выбор из нескольких возможных преобразований еще более усложняется, когда происходит вызов перегруженной функции. Если два или более преобразования обеспечивают подходящее соответствие, то преобразования считаются одинаково хорошими.

Например, могут возникнуть проблемы неоднозначности, когда перегруженные функции получают параметры, отличающиеся типами классов, которые определяют те же конструкторы преобразования:

ВНИМАНИЕ! ПРЕОБРАЗОВАНИЯ И ОПЕРАТОРЫ

Корректная разработка перегруженных операторов, конструкторов преобразования и функций преобразования для класса требует большой осторожности. В частности, если в классе определены и операторы преобразования, и перегруженные операторы, вполне возможны неоднозначные ситуации. Здесь могут пригодиться следующие эмпирические правила.

- Никогда не создавайте взаимных преобразований типов. Другими словами, если класс Foo имеет конструктор, получающий объект класса Bar, не создавайте в классе Bar оператор преобразования для типа Foo.
- Избегайте преобразований во встроенные арифметические типы. Но если преобразование в арифметический тип необходимо, то придется учесть следующее.
 - Не создавайте перегруженных версий тех операторов, которые получают аргументы арифметических типов. Если пользователи используют эти операторы, функция преобразования преобразует объект данного типа, а затем применит встроенный оператор.
 - Не создавайте функций преобразования больше, чем в один арифметический тип. Позвольте осуществлять преобразования в другие арифметические типы стандартным функциям преобразования.

Самое простое правило: за исключением явного преобразования в тип bool, избегайте создания функций преобразования и ограничьте неявные конструкторы теми, которые безусловно необходимы.

```
struct C {
    C(int);
    // другие члены
};
struct D {
    D(int);
    // другие члены
};
void manip(const C&);
void manip(const D&);
manip(10); // ошибка неоднозначности: manip(C(10)) или manip(D(10))
```

Здесь у структур `C` и `D` есть конструкторы, получающие значение типа `int`. Для версий функции `manip()` подходит любой конструктор. Следовательно, вызов неоднозначен: он может означать преобразование `int` в `C` и вызов первой версии `manip()` или может означать преобразование `int` в `D` и вызов второй версии.

Вызывающая сторона может устранить неоднозначность при явном создании правильного типа:

```
manip(C(10)); // ok: вызов manip(const C&)
```



Необходимость в использовании конструктора или приведения для преобразования аргумента при обращении к перегруженной функции — это признак плохого проекта.

Перегруженные функции и пользовательские преобразования

Если при вызове перегруженной функции два (или больше) пользовательских преобразования обеспечивают подходящее соответствие, они считаются одинаково хорошими. Ранг любых стандартных преобразований, которые могли бы (или не могли) быть обязательными, не рассматриваются. Необходимость встроеного преобразования также рассматривается, только если набор перегруженных версий может быть подобран и использован той же функцией преобразования.

Например, вызов функции `manip()` был бы неоднозначен, даже если бы один из классов определил конструктор, который требовал бы для аргумента стандартного преобразования:

```
struct E {
    E(double);
    // другие члены
};
void manip2(const C&);
void manip2(const E&);
// ошибка неоднозначности: применимы два разных пользовательских
// преобразования
manip2(10); // manip2(C(10) или manip2(E(double(10)))
```

В данном случае у класса `C` есть преобразование из типа `int` и у класса `E` есть преобразование из типа `double`. Для вызова `manip2(10)` подходят обе версии функции `manip2()`:

- Версия `manip2(const C&)` подходит потому, что у класса `C` есть конструктор преобразования, получающий тип `int`. Этот конструктор точно соответствует аргументу.
- Версия `manip2(const E&)` подходит потому, что у класса `E` есть конструктор преобразования, получающий тип `double` и возможность использовать стандартное преобразование для преобразования аргумента типа `int`, чтобы использовать этот конструктор преобразования.

Поскольку вызовы перегруженных функций требуют *разных* пользовательских преобразований друг от друга, этот вызов неоднозначен. В частности, даже при том, что один из вызовов требует стандартного преобразования, а другой является точным соответствием, компилятор все равно отметит этот вызов как ошибку.



Ранг дополнительного стандартного преобразования (если оно есть) при вызове перегруженной функции имеет значение, только если подходящие функции требуют того же пользовательского преобразования. Если необходимы разные пользовательские преобразования, то вызов неоднозначен.

Упражнения раздела 14.9.2

Упражнение 14.50. Представьте возможные последовательности преобразований типов для инициализации объектов `ex1` и `ex2`. Объясните, допустима ли их инициализация или нет.

```
struct LongDouble {
    LongDouble(double = 0.0);
    operator double();
    operator float();
};
LongDouble ldObj;
int ex1 = ldObj;
float ex2 = ldObj;
```

Упражнение 14.51. Представьте последовательности преобразования (если они есть), необходимые для вызова каждой версии функции `calc()`, и объясните, как подбирается наилучшая подходящая функция.

```
void calc(int);
void calc(LongDouble);
double dval;
calc(dval); // которая calc()?
```



14.9.3. Подбор функций и перегруженные операторы

Перегруженные операторы — это перегруженные функции. При выявлении, который из встроенных или перегруженных операторов применяется для данного выражения, используется обычный подбор функции (см. раздел 6.4, стр. 305). Однако, когда в выражении используется функция оператора, набор функций-кандидатов шире, чем при вызове функций, использующих оператор вызова. Если объект `a` имеет тип класса, то выражение `a sum b` может быть следующим:

```
a.operatorsym(b); // класс a содержит оператор sum как функцию-член
operatorsym(a, b); // оператор sum — обычная функция
```


В отличие от обычных вызовов функции, нельзя использовать форму вызова для различения функции-члена или не члена класса.

Когда используется перегруженный оператор с операндом типа класса, функции-кандидаты включают обычные версии, не являющиеся членами класса этого оператора, а также его встроенные версии. Кроме того, если левый операнд имеет тип класса, определенные в нем перегруженные версии оператора (если они есть) также включаются в набор кандидатов.

Когда вызывается именованная функция, функции-члены и не члены класса с тем же именем *не* перегружают друг друга. Перегрузки нет потому, что синтаксис, используемый для вызова именованной функции, различает функции-члены и не члены класса. При вызове через объект класса (или ссылку, или указатель на такой объект) рассматриваются только функции-члены этого класса. При использовании в выражении перегруженного оператора нет никакого способа указать на использование функции-члена или не члена класса. Поэтому придется рассматривать версии и функции-члены, и не члены класса.



Набор функций-кандидатов для используемого в выражении оператора может содержать функции-члены и не члены класса.

Определим, например, оператор суммы для класса `SmallInt`:

```
class SmallInt {
    friend
    SmallInt operator+(const SmallInt&, const SmallInt&);
public:
    SmallInt(int = 0);                // преобразование из int
    operator int() const { return val; } // преобразование в int
private:
    std::size_t val;
};
```

Этот класс можно использовать для суммирования двух объектов класса `SmallInt`, но при попытке выполнения смешанных арифметических операций возникнет проблема неоднозначности:

```
SmallInt s1, s2;
SmallInt s3 = s1 + s2; // использование перегруженного оператора +
int i = s3 + 0;        // ошибка: неоднозначность
```

Первый случай суммирования использует перегруженную версию оператора `+` для суммирования двух значений типа `SmallInt`. Второй случай неоднозначен, поскольку `0` можно преобразовать в тип `SmallInt` и использовать версию оператора `+` класса `SmallInt` либо преобразовать объект `s3` в тип `int` и использовать встроенный оператор суммы для типа `int`.



Предоставление функции преобразования в арифметический тип и перегруженных операторов для того же типа может привести к неоднозначности между перегруженными и встроенными операторами.

Упражнения раздела 14.9.3

Упражнение 14.52. Какой из операторов `operator+`, если таковые вообще имеются, будет выбран для каждого из следующих выражений суммы? Перечислите функции-кандидаты, подходящие функции и преобразования типов для аргументов каждой подходящей функции:

```
struct LongDouble {
    // оператор-член operator+ только для демонстрации;
    // обычно он не является членом класса
    LongDouble operator+(const SmallInt&);
    // другие члены как в р. 14.9.2 (стр. 741)
};
LongDouble operator+(LongDouble&, double);
SmallInt si;
LongDouble ld;
ld = si + ld;
ld = ld + si;
```

Упражнение 14.53. С учетом определения класса `SmallInt` на стр. 743 определите, допустимо ли следующее выражение суммы. Если да, то какой оператор суммы используется? В противном случае, как можно изменить код, чтобы сделать его допустимым?

```
SmallInt s1;
double d = s1 + 3.14;
```

Резюме

Перегруженный оператор должен либо быть членом класса, либо иметь по крайней мере один операнд типа класса. У перегруженных операторов должно быть то же количество операндов, порядок и приоритет, как у соответствующего оператора встроенного типа. Когда оператор определяется как член класса, его неявный указатель `this` связан с первым операндом. Операторы присвоения, индексирования, вызова функции и стрелки должны быть членами класса.

Объекты классов, которые перегружают оператор вызова функции, `operator()` называются “объектами функций”. Такие объекты зачастую используются в комбинации со стандартными алгоритмами. Лямбда-выражения — это отличный способ определения простых классов объектов функции.

Класс может определить преобразования в или из своего типа, которые будут использованы автоматически. Неявные конструкторы, которые могут быть вызваны с одним аргументом, определяют преобразования из типа параметра в тип класса; операторы неявного преобразования определяют преобразования из типа класса в другие типы.

Термины

Объект функции (function object). Объект класса, в котором определен перегруженный оператор вызова. Объекты функций применяются там, где обычно ожидаются функции.

Оператор преобразования (conversion operator). Оператор преобразования — это функция-член, которая осуществляет преобразование из типа класса в другой тип. Операторы преобразования должны быть константными членами их класса. Такие функции не получают параметров и не имеют типа возвращаемого значения. Они возвращают значение типа оператора преобразования. То есть оператор `operator int` возвращает тип `int`, оператор `operator string` — тип `string` и т.д.

Перегруженный оператор (overloaded operator). Функция, переопределяющая значение одного из встроенных операторов. Функция перегруженного оператора имеет имя `operator` с последующим определяемым символом. У перегруженных операторов должен быть по крайней мере один операнд типа класса. У перегруженных операторов тот же приоритет, порядок и количество операндов, что и у их встроенных аналогов.

Пользовательское преобразование (user-defined conversion). Синоним термина преобразование типа класса.

Преобразование типа класса (class-type conversion). Преобразования в или из типа класса определяются конструкторами и операторами преобразования соответственно. Неявные конструкторы, получающие один аргумент, определяют преобразование из типа аргумента в тип класса. Операторы преобразования определяют преобразования из типа класса в заданный тип.

Сигнатура вызова (call signature). Представляет интерфейс вызываемого объекта. Сигнатура вызова включает тип возвращаемого значения и заключенный в круглые скобки разделяемый запятыми список типов аргументов.

Таблица функций (function table). Контейнер, как правило, карта или вектор, содержащий значения, позволяющие выбрать и выполнить функцию во время выполнения.

Шаблон функции (function template). Библиотечный шаблон, способный представить любой вызываемый тип.

Явный оператор преобразования (explicit conversion operator). Оператор преобразования с предшествующим ключевым словом `explicit`. Такие операторы используются для неявных преобразований только в определенных условиях.

